

Apache Kafka

Complete Detailed Notes — From Beginner to Advanced

An in-depth, example-rich study guide covering asynchronous processing patterns, message queues, event streaming, and the complete internal working of Apache Kafka — topics, partitions, brokers, replication, offsets, consumer groups, delivery guarantees, backpressure, metadata, the full producer/consumer write & read path, and Kafka vs. RabbitMQ / Pub-Sub — with worked numeric examples, code snippets, and 20+ diagrams.

Compiled & expanded from a detailed video lecture on Kafka

Table of Contents

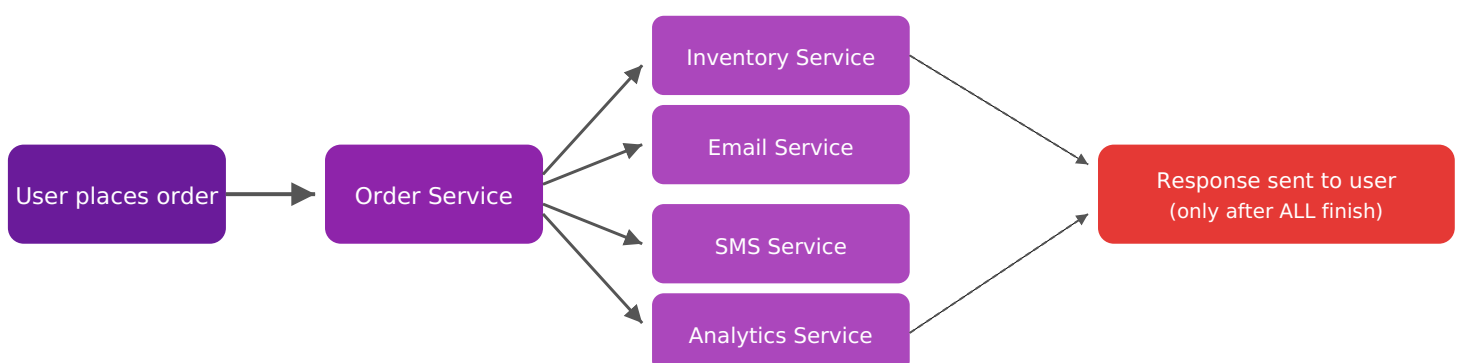
1. The Problem: Why Asynchronous Processing?
2. Seven Ways to Do Asynchronous Processing (with code & examples)
3. Message Queue vs Event Streaming — In Depth
4. Is Kafka a Message Queue or an Event Stream?
5. Understanding Kafka with a Real-Life Analogy (Sports)
6. Core Kafka Concepts — Full Technical Deep Dive
 - a. Topics
 - b. Partitions (with worked hashing examples)
 - c. Brokers & Kafka Cluster
 - d. Consumers & Consumer Groups (full rules)
 - e. Offsets (with full crash/recovery walkthrough)
7. How Consumers Actually Read Data (Poll / Long-Poll Model)
8. Replication & the Leader-Follower Model (in depth)
9. Kafka Metadata & the Controller (full worked example)
10. End-to-End Kafka Flow — Every Step, With Numbers
11. Handling Wrong / Bad Data in Kafka
12. Delivery Guarantees — At-most / At-least / Exactly-once (full walkthroughs)
13. Backpressure — The Full Explanation
14. Frequently Asked Questions — 20+ Q&A Deep Dive
15. Why Kafka Handles Massive Reads & Writes
16. Kafka vs Pub/Sub — Full Comparison
17. Kafka vs RabbitMQ (Event Stream vs Message Queue) — Full Comparison
18. Kafka in Real System Design (YouTube Upload Pipeline, Worked End-to-End)
19. Code Snippet Appendix
20. Final Summary Cheat-Sheet

1. The Problem: Why Asynchronous Processing?

1.1 The Amazon Order Example

Imagine a system like **Amazon**. When a user places an order, a lot of work has to happen behind the scenes before the "order placed" flow is truly complete:

- Save the order in the database
- Reduce inventory count — e.g. if there were 30 units of an item, placing one order should reduce the count to 29, since this number represents how much stock is left in the system
- Send a confirmation email to the user
- Send an SMS
- Notify the warehouse so the item can be picked and packed
- Generate an invoice
- Update analytics dashboards
- ...and depending on the system, possibly many more steps



1.2 The Naive (Synchronous) Approach and Its Problems

A brute-force approach would be to do everything **synchronously**: order → order service → inventory microservice → email microservice → SMS microservice → analytics microservice, one after another, and only send the API response once *every single step* has completed. This causes three serious problems:

1. **The user has to wait for everything to finish.** Since the response is only returned after all steps complete, there's a huge amount of added latency. The response becomes slow, and it's a bad user experience because the user has no idea (and shouldn't have to know) about all this internal work. From the user's point of view, it's a simple system: "I placed an order, tell me it succeeded." They don't care about inventory updates, emails, or analytics happening on the backend.
2. **The slowest microservice determines the overall speed.** If any one of these downstream microservices responds slowly, the entire API response becomes slow, and the end user is impacted — even if their actual order-critical work finished instantly.
3. **A single failure kills the whole request.** If any of the services is down, the order simply fails, because the response is only sent when all steps pass. This makes the system fragile and drastically hurts user experience — the user can't even place a basic order because some unrelated internal service (like the analytics pipeline) is temporarily broken.

1.3 Real-Life Analogy — Ordering Food

Picture a food counter with a queue. You reach the front, place your order (say, a pizza), and now you must wait right there because your food is being prepared. If you leave, you lose your place and have to rejoin a potentially huge queue. This is a terrible experience — the customer is **blocked**, even though the actual cooking has nothing to do with them directly.

The fix real food counters use: as soon as you order, they give you a **token or order number** (e.g. "Order #32"). There might be a display board that later shows "Order 32 ready," or they might text or call you. The moment you have that token, you're free — you can sit down, watch TV, talk to friends, or leave and come back. You are **not blocked**.

This is exactly the shift we want technically: once the **order microservice** finishes its essential work, the user should immediately get a response ("Order placed successfully!"), and everything else (inventory update, email, SMS, warehouse notification, invoice, analytics) should happen **asynchronously**, in the background, without blocking the user.

1.4 Technical Example — YouTube Video Upload

When a user uploads a video to YouTube, apart from storing the raw video file, a lot of additional work is required:

- Transcoding the video into multiple resolutions (144p, 480p, 720p, 1080p, 4K)
- Running copyright checks
- Generating automatic thumbnails
- Updating analytics
- ...and many more steps depending on the platform

Doing all of this synchronously means the API response depends on every one of these microservices completing — and if any one of them fails, the user simply cannot upload their video, and they're stuck waiting far longer than necessary. The fix: acknowledge the upload as soon as the raw video is safely stored, and let everything else — transcoding, copyright checks, thumbnail generation, analytics — happen asynchronously in the background.

Key takeaway: Whenever there is work that does not need to block the caller's response, that work should be moved off the critical path and processed **asynchronously**. The remaining question is simply: *how* do we implement that asynchronous processing? That's what the rest of these notes are about.

2. Seven Ways to Do Asynchronous Processing

Before jumping into Kafka, it's worth seeing the **full landscape** of options available for doing work asynchronously — what their trade-offs are, and when each one makes sense. You don't need to memorize every detail here; the goal is to build enough context that Kafka's specific role becomes obvious later.

2.1 Background Threads / Executors

The simplest option: create a background thread or an executor *inside the same service*. The main flow hands off a task, and the background thread picks it up and performs it.

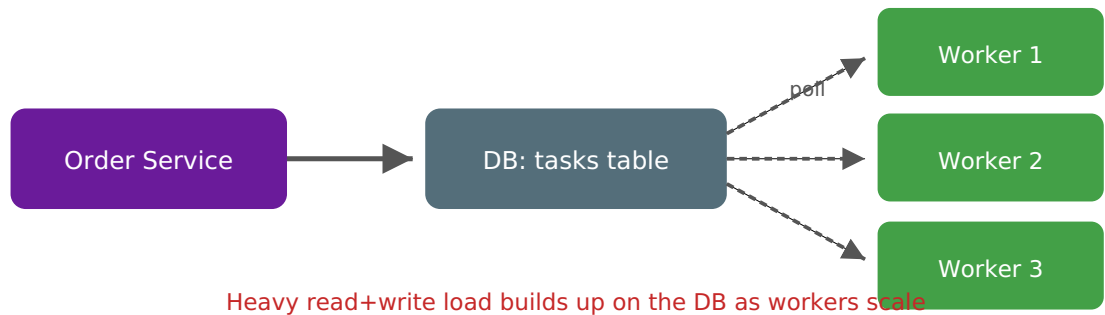
```
executor.submit(() -> {
    // perform the async action here, e.g. send email, update cache, etc.
});
```

Pros	Cons	When to use
Very simple, no infra needed, low latency, good for small apps	Job is lost if the service crashes (no durability), no retries, not scalable across machines (tied to a single process/machine)	Single-server apps, non-critical tasks, side projects / POCs — not suited for real distributed system design

In high-level system design we always think in terms of **distributed systems** — multiple servers, multiple machines. Because an executor lives inside one service/machine, this approach is unreliable, not durable, and not fault-tolerant at that scale. It's more of a low-level design tool.

2.2 Database Polling

Push tasks into a database "tasks" table from the producing service; separate worker processes continually **poll** that table, pick up unprocessed rows, and process them asynchronously.



Pros	Cons	When to use
Durable, because a DB has persistent storage; easy to implement; simple retry logic	Polling the DB repeatedly is inefficient; you're adding a lot of write operations (inserting tasks) <i>and</i> a lot of read operations (polling) — putting heavy load on the database, which can even bring it down; scaling workers is painful; ordering is hard to maintain	Only for low-throughput systems — not scalable when we talk about high-throughput needs

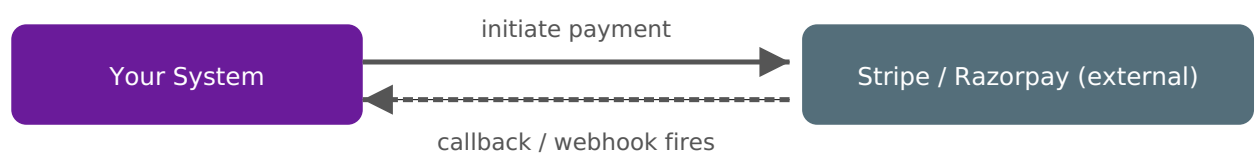
2.3 Cron Jobs / Scheduled Workers

A job or worker runs at fixed intervals — e.g. every 5 minutes or every hour, depending on the task. Typical examples: sending reminder emails, cleaning up old data, generating daily reports.

Pros	Cons	When to use
Extremely simple, predictable execution, no dependency on any external event	Not real-time — if the job ran at 8:00 PM and something happens at 8:25 PM, it has to wait until the next scheduled run (e.g. 9:00 PM); this introduces delay, so it's a poor fit for user-triggered, time-sensitive actions	Time-based async work, batch processing (everything that happened between two runs gets processed together), and general maintenance tasks. Cron jobs are time-driven, not event-driven .

2.4 Webhooks

A webhook is when **your system calls someone else's endpoint** asynchronously. There's an external system or third-party integration that exposes an endpoint; you make a request and then wait for a callback from that external endpoint. A classic example is designing a **payment system**: you initiate a payment through a third-party provider like Razorpay or Stripe, and your system polls / waits for a callback regarding that specific transaction.



Pros	Cons	When to use
Simple, doesn't require extra infra, good for integrations	Network failures can occur since it happens over the network; this brings retry complexity; can create tight coupling between systems	Third-party integrations — payment gateways (Stripe, Razorpay), GitHub-style event notifications. Overall: async, but not reliable without retries .

2.5 Serverless / Task Services (AWS Lambda, Cloud Tasks, Azure Functions)

These are cloud-native ways of running async work. A simple illustration: your product normally sees average load X, but during a festival sale, load can spike far beyond that. One option is manually increasing the number of servers ahead of the sale and decreasing them afterward — but that only works for *predictable* spikes. For unpredictable spikes, cloud providers offer **auto-scaling**: the number of servers adjusts automatically based on load, and you simply **pay for what you use**.

Pros	Cons	When to use
Auto-scaling, no server management, pay-per-use	Cold starts, vendor lock-in, execution limits	Cloud-native systems with spiky/unpredictable load

2.6 Message Queue

This is essentially the classic **producer-consumer problem**. A **producer** represents any component that produces a task ("this needs to be done"). A **consumer** is a component that picks up and performs that task. Crucially, this has to happen *asynchronously* — it's not that the moment a producer asks for a task, a consumer immediately grabs it (that would just be synchronous processing all over again).

A **message queue** sits in between as **temporary storage**. Whatever task needs to be performed is added to the queue, and a consumer eventually consumes it from there. Because the queue holds the task temporarily, the producer doesn't have to wait around — it can move on immediately. Once a consumer finishes processing a task, that task is **removed** from the queue. It is temporary storage, not permanent storage — it exists purely to enable asynchronous processing.

2.7 Event Streaming

As the name suggests, this is based on **events**. Events are published, and **many consumers** can consume those events. This might sound similar to a message queue at first glance, but the differences are significant — covered in full detail in Section 3.

2.8 Comparing All Seven Options

Option	Reliable?	Scalable?	Real-time?
In-memory threads	✗ No — job lost if the machine goes down	✗ No — tied to one machine, not distributed	✔ Yes
Database polling	✔ Yes — DB is persistent storage	✗ No — heavy read+write load on the DB	Depends on poll frequency
Cron jobs	✔ Yes — every fixed interval covers whatever happened	Limited — not meant to scale for user-triggered work	✗ No — bound to fixed intervals
Webhooks	⚠ Only with retries built in	Moderate	✔ Yes
Serverless	✔ Yes	✔ Yes (auto-scaling)	✔ Yes
Message Queue	✔ Yes — task persists in the queue until consumed	✔ Yes	✔ Yes
Event Streaming	✔ Yes — events persist in the log for the retention period	✔✔ Excellent (this is why Kafka scales so well)	✔ Yes

3. Message Queue vs Event Streaming — In Depth

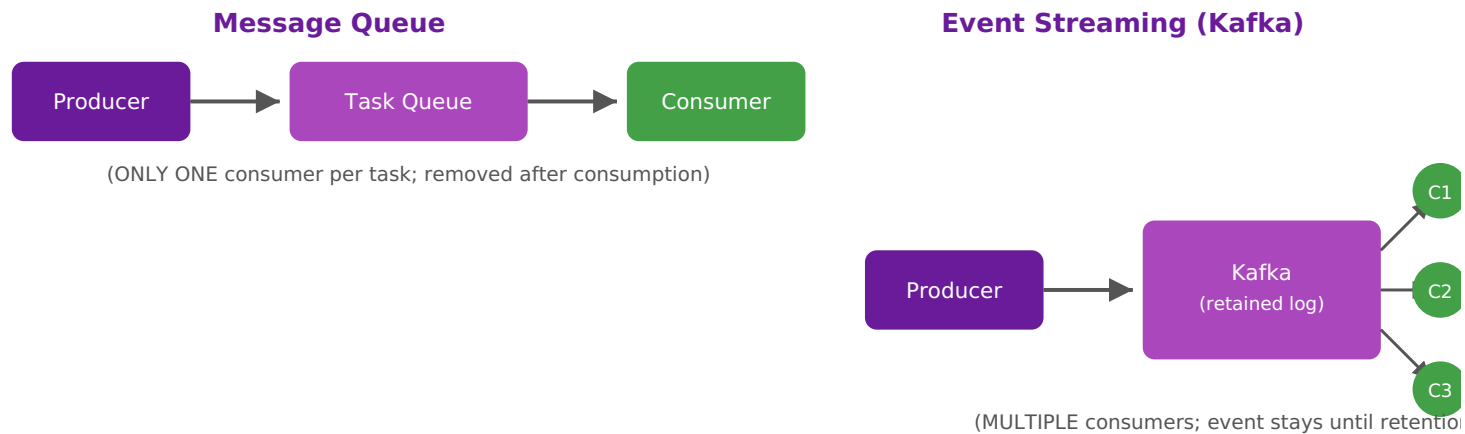


Fig 3.1 — Message queue (one consumer, message removed after use) vs Event stream (many independent consumers, event retained).

3.1 What Gets Stored

In a **message queue**, you store a **task** — think of it like a to-do list. Each message is a job that needs to be performed exactly once, and once performed, you simply forget about it. Example: "send an email for order ID 501" — once the email is sent, that task is removed from the queue.

In **event streaming**, you don't store tasks — you store **events**. An event stream behaves like a **log file** or a document: you simply **append** to the end, to keep it fast and scalable. This means the data inside must be **immutable**. If you tried to store mutable *state* (like an order's current status, which moves from "placed" → "shipped" → "delivered" → possibly "returned"), you'd need to keep updating it — but Kafka doesn't do updates. Instead, each event is treated as a **fact** that already happened (e.g. "order was shipped at this timestamp") and is never modified — you simply keep appending new facts as they occur.

3.2 Number of Consumers — The Biggest Difference

In event streaming, **multiple consumers** can read the same event. In a message queue, **only one** consumer can consume a given message. This single distinction is the reason Kafka's power comes from being an event stream rather than a message queue.

3.3 Message/Event Lifetime

In a message queue, a task stays only until it's consumed — the moment the (single) consumer processes it, it's removed. In event streaming, the event stays for the configured **retention period** — commonly somewhere between **7 and 28 days** — completely **independent** of whether it has been consumed. Even if every consumer has already read an event, it still physically remains in Kafka until the retention window expires. Conversely, if *no* consumer ever reads it, it still gets deleted once the retention period ends — Kafka does not wait around for consumption.

Aspect	Message Queue	Event Streaming
What is stored	Tasks / jobs ("to-do list")	Events (immutable facts, append-only log)
Number of consumers	Only one consumer per message	Multiple consumers can read the same event
Message lifetime	Removed immediately after consumption	Stays for the retention period (e.g. 7-28 days), independent of consumption
Replay	✗ Not possible (already deleted)	✓ Possible (event still exists until retention expiry)
Consumer state	Managed by the queue itself	Managed by each consumer (via offsets)
Durability	Lower — gone right after consumption	Higher — survives well beyond consumption
Example	RabbitMQ	Apache Kafka

Three major differences to remember: (1) task vs event, (2) one consumer vs many consumers, (3) removed on consumption vs retained for a fixed period. Event streaming is fundamentally more **durable** than a message queue.

4. Is Kafka a Message Queue or an Event Stream?

For a long time it's easy to assume Kafka is "just a message queue" — a very natural, but incorrect, assumption to make before studying Kafka's internals in depth.

Kafka can technically act as both, but the vast majority of the time it behaves as an **event streaming platform** — because multiple independent consumers can read the same event, and events are retained for a configurable period rather than deleted immediately after consumption.

Kafka only "looks like" a message queue in the narrow case where a topic's partition has exactly one consumer group with one active consumer reading it — see Section 15.3 for that edge case, explained with a diagram.

5. Understanding Kafka with a Real-Life Analogy (Sports)

Imagine a system that tracks events for many different sports: football/FIFA, cricket, basketball, lawn tennis, F1 racing, and more. This system *is* Kafka.

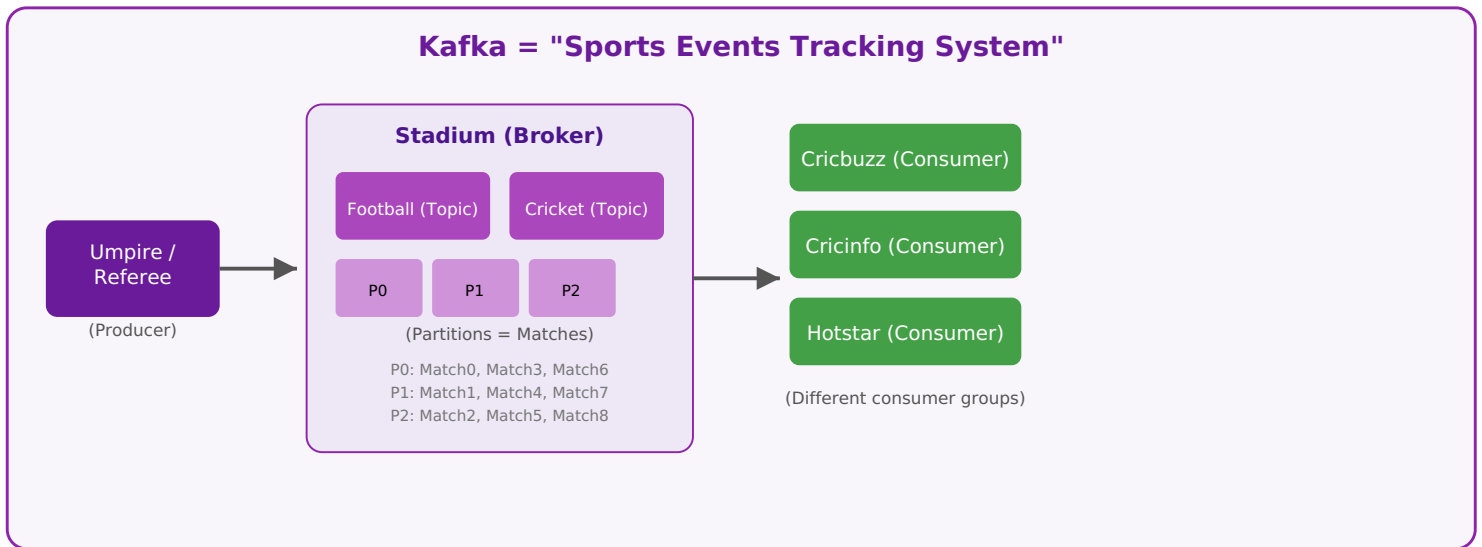


Fig 5.1 — Sports analogy: Stadium = Broker, Sport = Topic, Match = Partition, Umpire = Producer, Media apps = Consumers.

5.1 Topic vs Event — Not the Same Thing

Applications like Cricbuzz or Cricinfo only care about **cricket** events — they don't want football or tennis events mixed in. That's exactly what a **topic** solves: a topic is a **set of similar, related events**. For example, "football events" as a topic would contain events like: goal scored, match started, foul committed. "Cricket events" as a topic would contain: boundary hit, six hit, wicket gone, wide ball, no ball, etc.

Q: Is a Topic the same as an Event?

No. A topic is a broad category that can contain **multiple different event types** — it's not restricted to a single kind of event. All of the related events above (goal, foul, match-started) belong to the single topic "football events."

Example of an Event's Structure

```
{
  "topic": "football_events",
  "event": "goal_scored",
  "player": "Messi",
  "matchId": "M2021",
  "timestamp": "2026-09-11T10:15:00Z"
}
```

A single event carries multiple fields: which event happened, extra context (like who scored), a match/entity ID, a timestamp, and possibly more.

5.2 Where Are Events Physically Stored?

We know events live "inside Kafka" logically — but where are they *physically* stored? Just like in system design a "server" is physically a machine, in Kafka the physical storage answer is: **broker and partition**.

5.3 Mapping the Analogy to Kafka Terms

Real-Life Concept	Kafka Term	Explanation
-------------------	------------	-------------

The overall system tracking all sports	Kafka	Contains all events, across all categories
A sport (football, cricket, tennis)	Topic	A logical category / stream of related events
A specific event (goal scored, six hit, foul)	Event	A single fact — a topic contains many different event types
The stadium where matches happen	Broker	A server/machine that physically stores and serves data
All stadiums together	Kafka Cluster	Multiple brokers combined, to avoid single point of failure
An individual match (India vs Pakistan)	Partition	The physical storage unit inside a broker holding actual events
The umpire / referee	Producer	Produces the events (out, six, goal, foul, etc.)
Apps like Cricbuzz, ESPN, Hotstar	Consumer	Reads and processes events
A company like "Cricbuzz" (with employees)	Consumer Group	A logical group of consumers working together

5.4 Broker: Why Multiple, and Why "Cluster"

A **broker** is a machine or server inside which actual data physically lives. Kafka contains **multiple brokers** to avoid a **single point of failure** — if there were only one broker and its machine went down, the entire system would go down, which is unacceptable for a distributed system. Combined, all these brokers/machines are called a **Kafka cluster**.

5.5 Why We Need Partitions Even Though We Have Topics — The Suitcase Analogy

A topic is just a **logical** partitioning of data — it is not physical. The *physical* representation of the distribution of data happens via **partitions**. Here's a simple, memorable way to see why partitions matter:

Worked analogy — the trolley bag: Imagine packing a trolley bag with clothes directly, with no organization. If it exceeds the 15 kg (33 lb) weight limit and you have a second bag, one option is to manually pull clothes out one by one and move them to the other bag — tedious. A far better approach is to pack clothes into several **small bags** inside the trolley first. Now, if you need to rebalance the load between two trolleys, you just pick up a whole small bag and move it — much simpler. Those small bags are exactly like **partitions**: physical units of data that can be moved, scaled, and load-balanced far more easily than the whole (logical) collection.

Similarly, a single broker/machine could otherwise contain a jumbled mess of events from many different topics with no organization. Partitions (combined with topics) keep this sorted: **topic = logical distribution, partition = physical distribution**, where the actual data lives.

5.6 A Broker Can Hold Multiple Partitions — Even From Different Topics

Going back to the analogy: the same stadium (broker) can host different sports on different days — a football pitch one day, a cricket match another. Likewise, a single Kafka broker (machine/server) can physically host multiple partitions, and those partitions don't all have to belong to the same topic. This directly answers two important questions:

- Q: Can a broker contain multiple partitions? Yes.
- Q: Can a broker contain events of different topics? Yes — a broker can host partition "cricket-events-P0" and partition "football-events-P0" side by side, just as a stadium can host both a cricket match and a football match (on different occasions).

5.7 One Partition Can Cover Multiple Matches (via Hashing)

Suppose the "cricket events" topic has three partitions (P0, P1, P2), and there are many simultaneous World Cup matches: Match 0 = India vs Pakistan, Match 1 = Australia vs England, and so on, up to Match 8. Kafka doesn't dedicate one partition per match — instead, it distributes matches across partitions using **hashing**, very similar to load-distribution techniques used elsewhere in system design.

Partition	Matches Handled
P0	Match 0, Match 3, Match 6
P1	Match 1, Match 4, Match 7
P2	Match 2, Match 5, Match 8

This works out to a simple modulus hash: `partition = matchId % 3`. So Match 0 → 0 % 3 = 0 → P0. Match 1 → 1 % 3 = 1 → P1. Match 3 → 3 % 3 = 0 → P0 again. And so on. Because of partitioning, multiple consumers can independently consume from P0, P1, and P2 in parallel — this is central to how Kafka achieves both distribution and asynchronous processing at scale.

5.8 Partition Failure — Kafka Already Handles It

Just like brokers need redundancy to avoid single points of failure, so do partitions. Kafka assigns a **replication count** to each partition (e.g. 3), meaning that many copies of that partition exist across *different* brokers. If the "leader" copy goes down, one of the replicas is promoted to take over — this is explained fully in Section 8 (Replication & Leader-Follower Model).

5.9 Ordering of Events

Consider cricket: a player can't get "out" before hitting a six that leads to it — order matters. Because all events tied to the same match ID always hash to the *same* partition, and Kafka only ever **appends** at the end of a partition's log, the events for that specific match arrive and are stored in the correct order within that partition.

But across different partitions (P0 vs P1), ordering is NOT maintained. These operate independently — syncing order across partitions would require constant cross-machine coordination, which would slow the system down and reintroduce tight coupling. Since P0 might hold Match 0's events and P1 holds Match 1's events, there's no need to order events between two completely different matches anyway.

If the leader partition for a match (say P0) goes down, one of its replicas is promoted as the new leader, and all reads/writes continue from there — ordering for that match is preserved because all its events still land in the "logical" P0 (now served by a different physical replica).

What if an event doesn't carry any ID to hash on? Then ordering simply isn't something we care about for that event, and Kafka can assign it to a partition using a **round-robin** approach instead.

5.10 Consumer Groups — Why They're Needed

Consider "Cric Update," an app giving live cricket score updates — this is a **consumer** reading from the "cricket events" topic. Suppose this company has three employees, all tracking match zero. If an event like "out" happens, and *all three* employees pick it up and independently update their own database, you get duplicated, wasted, and potentially conflicting effort. That's not productive, and it's also error-prone.

What we actually want: this group of employees should act as **one coordinated unit** where only one of them handles a given partition at a time — while still allowing the group as a whole to divide and conquer across multiple partitions (multiple matches) in parallel. This is exactly what a **consumer group** gives you.

The golden rule of consumer groups: only **one** consumer inside a consumer group may consume from a given partition at any time. But a single consumer *can* consume from **multiple** partitions — that's perfectly fine and is, in fact, the beauty of Kafka's scalability.

So: Employee 1 (E1) can consume from Partition 0, E2 from Partition 1, E3 from Partition 2 — all three covering different matches in parallel, maximizing productivity, with zero duplicate effort. And a *different* consumer group (say, "Cricket Tracker," a different application) can *also* independently consume from Partition 0 at the same time — because it's a different company/consumer group. That's the beauty of Kafka acting as an event stream: **multiple different consumer groups** can all read the same partition, each fully independently.

Q: Can a consumer exist without belonging to a consumer group?

No — in Kafka, a consumer **cannot exist without a consumer group**. If you don't explicitly define one, Kafka creates a default one for you behind the scenes. Think of it like employment: an employee is always part of some company. The company is the consumer group; the employee is the consumer.

6. Core Kafka Concepts — Full Technical Deep Dive

6.1 Topic

- A topic is a **category or stream of events** (e.g. `order_events`, `user_activity`, `payment_events`).
- A topic is a **logical** construct — it does not physically store data itself; topics do not store data, partitions do.
- A topic can contain **multiple different event types**. For example, an `order_events` topic might carry both an `order_created` event (with `orderId`, `userId`, `amount`) and an `order_cancelled` event (with `orderId`, `userId`) — different event shapes, same topic.

YouTube example: A topic could be `system_design_videos`, containing events like `video_uploaded`, `video_viewed`, `video_liked`, `video_commented` — all different event types published to the same topic.

6.2 Partition

Topics are just logical grouping; the actual physical storage happens in **partitions**. A broker can host multiple partitions, and each partition physically stores a sequence of events on disk, written **sequentially**.

Why do we need partitions if we already have topics?

The Suitcase Analogy (recap)

Think of a topic as a **folder name** and partitions as the **actual files** inside. Instead of throwing all clothes loosely into one suitcase (a single giant log that folder. You cannot write data into a folder — you write into files. file — not scalable, single point of failure), you use several small bags. Similarly, Kafka needs a physical place (partition) to write events into. If a (partitions) inside the suitcase — making it easy to move, scale and topic had just *one* giant log file, you'd get zero scalability, zero parallelism, balance load between machines. and a single point of failure — that single file dying takes the whole topic down.

Key Rules about Topics & Partitions

- A **topic** can contain **multiple partitions**.
- A **partition** belongs to **exactly one topic** — never shared/mixed across topics.
- Each partition is an **ordered, append-only log** — events are written sequentially, one after another, to disk.
- Ordering is guaranteed **within** a partition, but **not across** partitions of the same topic.
- A broker can hold **multiple partitions**, and those partitions may belong to **different topics**.
- Partitions provide: **scalability, parallel processing, load balancing, and high throughput**.

Topic = Logical | Partition = Physical

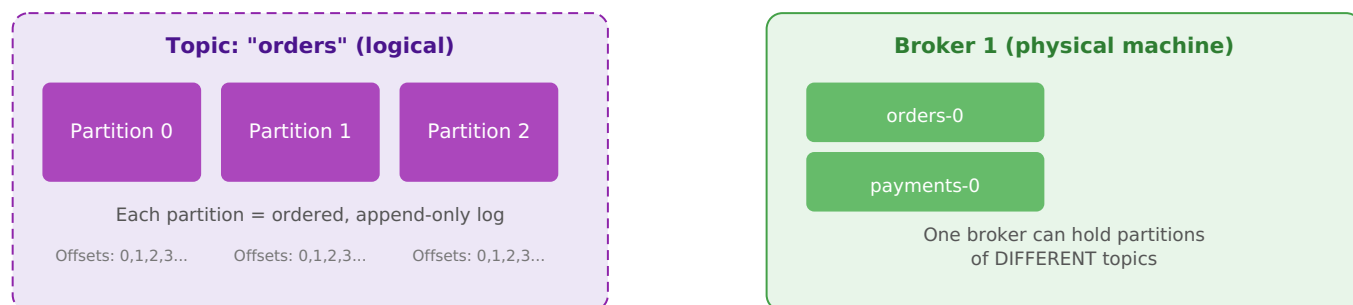


Fig 6.1 — A topic is split into multiple physical partitions; a single broker can host partitions from different topics.

Ordering & the Hashing Function — Full Worked Example

Kafka assigns which partition an event goes to using a hash of a **key** present in the event (e.g. match ID, order ID): `partition = hash(key) % num_partitions`. Because the same key always hashes to the same partition, all events for that key land in the same partition and stay in order (since Kafka only ever appends at the end).

Worked example: Suppose `order_id = 123`. We pass 123 into a hash function and (say) get the value `4`. If there are 3 partitions, we compute `4 % 3 = 1` → this event goes to **Partition 1**. Crucially, *every* event tagged with order ID 123 will always hash to the same value and always land in Partition 1 — which is exactly how ordering is preserved for that specific order, even though ordering is not preserved globally across the whole topic.

If no key is present, Kafka instead assigns partitions in a simple **round-robin** fashion — this is fine because, without a key, there's typically no ordering requirement to preserve in the first place.

Q: How many partitions should I choose at the start? It depends on expected throughput (events/sec), the maximum number of consumers you'll ever need (since consumers parallelize by reading different partitions), broker capacity, and future growth. Rule of thumb: **Small systems:** 3–6 partitions · **Medium systems:** 12–24 partitions · **Large systems:** 50+ partitions.

Q: Can I increase partitions later if data volume grows? Yes, technically — but it's risky. Because the hash function's modulus changes (e.g. from `% 3` to `% 4` after adding a 4th partition), a key that used to map consistently to Partition 0 might now map to the new Partition 3 — breaking the previously guaranteed ordering for that key going forward. Note: this issue is *not* the same as a partition going down — if a leader partition fails, a *replica* takes over as the new leader, and the total partition count (and hence the hash function) stays exactly the same, so ordering is unaffected in that scenario.

Creating a Topic (CLI Example)

```
kafka-topics.sh --create \
  --topic orders \
  --partitions 6 \
  --replication-factor 3
```

This single command creates a topic named `orders` with **6 partitions** and a **replication factor of 3** (meaning 2 extra copies beyond the original, for a total of 3 copies of each partition, spread across different brokers).

Topics can be created by an operator, developer, or platform team via API/CLI (the most common and recommended approach), or auto-created automatically by Kafka — though auto-creation is **not recommended** for serious production setups, since you lose control over partition count and replication settings.

6.3 Brokers & Kafka Cluster

- A **broker** is simply a server/machine that stores data and serves read/write requests.
- Kafka has **multiple brokers** to avoid a single point of failure — with just one broker, that machine dying would take the whole system down.
- All brokers together form the **Kafka Cluster**.
- Kafka scales horizontally by **adding more brokers**.
- Overall, Kafka can be thought of as a **distributed log file** — you only ever append events at the end; there's no update or delete of individual records (more on this in Section 10).

6.4 Consumers & Consumer Groups — Full Rules

Golden Rule: Within a single consumer group, only **one** consumer can consume from a given partition at a time. But consumers from **different** consumer groups *can* all read from the same partition independently — that's what makes Kafka a true event-streaming platform.

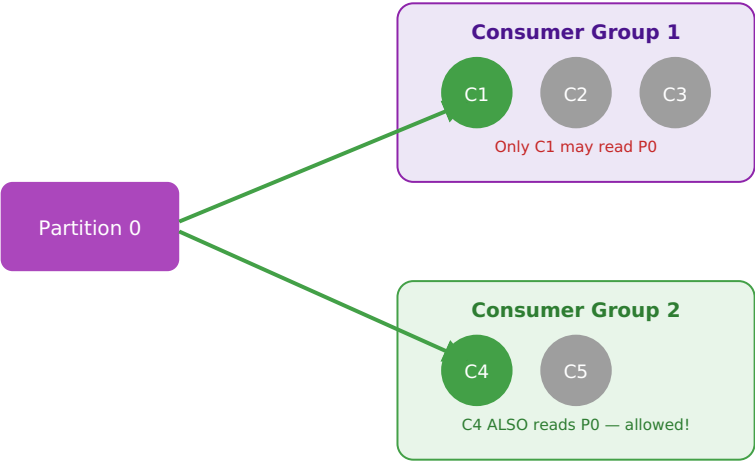


Fig 6.2 — Only one consumer per consumer group per partition, but multiple different consumer groups can read the same partition.

- A consumer **cannot exist without a consumer group** — Kafka auto-assigns one if you don't specify it explicitly (e.g. `CG1`, `CG2` by default).
- One consumer **can** consume from multiple partitions, and even multiple topics — including via **regex pattern** subscriptions like `orders.*`, or subscribing to literally every topic.
- A single consumer group commonly has **fewer consumers than partitions** (very typical in practice, since microservices/consumer groups are limited but partition counts can be large) — in that case, one consumer simply consumes from **multiple** partitions of the same topic.
- Consumer groups enable both **parallelism** (many partitions processed at once, across consumers) and **safety** (no duplicate work within the group, since only one consumer per partition per group).
- A consumer that is itself consuming an event can **also act as a producer** for a downstream event — e.g. a transcoding service consumes `video_uploaded` and later produces `video_transcoded`.

Example: Consumer Group with 6 Partitions, 2 Consumers

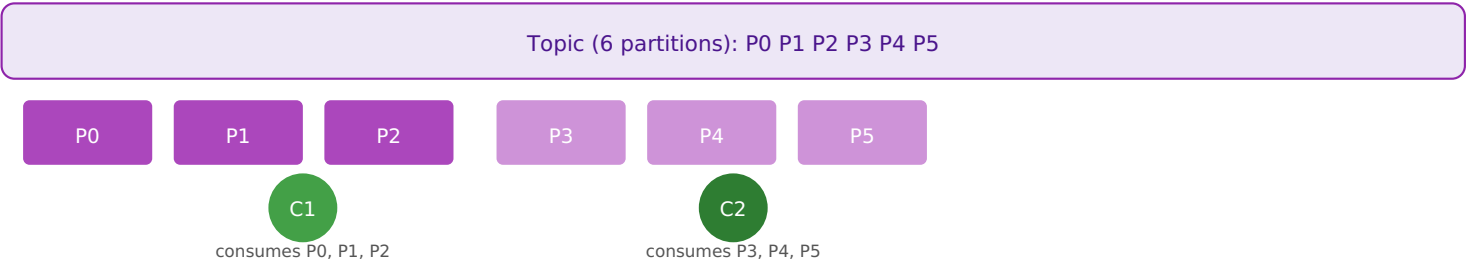


Fig 6.3 — A consumer can consume from multiple partitions of the same topic; only the "no two consumers on the same partition, same group" rule applies.

6.5 Offsets — Full Explanation with Crash/Recovery Walkthrough

An **offset** is a monotonically increasing number that uniquely identifies a record **within a partition**. Offsets are unique per partition — Partition 0 and Partition 1 can both have an event at offset "5"; that's fine because they're independent logs.

Offset scope: Offsets are tracked **per consumer group, per partition**. This is exactly what allows different consumer groups (or different consumers reading different partitions) to progress completely independently of one another — like independent microservices (e.g. a "payment service" and a "profile service") each consuming at their own pace, with their own requirements.

Partition 1 — append-only log of events



Fig 6.4 — Two consumers reading the same partition independently, each tracking its own offset.

So what exactly does an offset *mean*? If C1 says its offset for Partition 1 is **20**, it means C1 has already successfully processed all events **up to 20**, and now wants events **from 20 onward**. If C3 says its offset is **34**, it already has everything before 34 and wants events from 34 onward. Put simply: **an offset is a bookmark** — "I have successfully processed all messages before this point."

Where Are Offsets Stored?

Offsets are stored inside Kafka itself, in an **internal topic** called `__consumer_offsets` (referred to in the lecture simply as "consumer offsets"). Each record there is keyed by **(consumer group, topic, partition)** and its value is the last committed offset. This makes offset-tracking durable, replicated, and able to survive consumer crashes — because Kafka itself is durable.

```
Key: (group = "payment-service", topic = "orders", partition = 0)
Value: committed_offset = 125
```

Only **one record** is maintained per (partition, consumer group) pair — it's not that Kafka keeps a growing history of every offset value ever committed; it simply updates the single existing value (e.g. from 20 to 30) each time a new commit happens. So this internal topic doesn't grow unbounded, and in many Kafka deployments, this particular internal topic is treated as exempt from normal retention-period cleanup, precisely because it just tracks the latest bookmark.

Full Walkthrough — Consumer Crash and Recovery Using Offsets

This is one of the most important flows to understand deeply:

- 1. A consumer group fetches its last committed offset for a partition from `__consumer_offsets` — say it gets **20**.
- 2. It polls Kafka and fetches events **20 through 30** (say 10 events).
- 3. While processing these events, the consumer group's service **crashes** before it finishes.
- 4. Because the offset was *never updated/committed* past 20, when the service comes back up and asks Kafka for its offset again, it will **still get 20** — not 30.
- 5. So it re-fetches from 20 onward, tries to perform the task again — this is safe because events with retention (e.g. 28 days) are still sitting there in Kafka, untouched, regardless of the earlier crash.
- 6. Once it fully and successfully completes processing (say the batch from 20 to 30), it's the **consumer group's own responsibility** to commit the new offset — updating the value in `__consumer_offsets` from 20 to **30**.
- 7. Next time it asks for its offset, it will now correctly get **30** and continue fetching events from 30 onward.

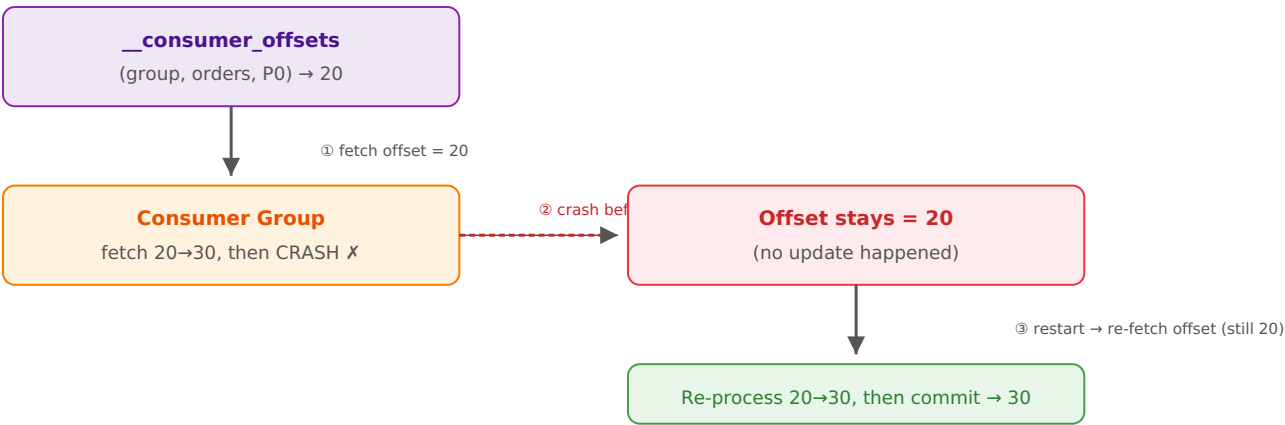


Fig 6.5 — Crash-and-recovery flow: the offset in Kafka only advances after a successful commit, so a crash simply causes safe re-processing.

Offset Commit Strategies

Strategy	Description	Risk
Auto-commit	Kafka periodically commits the offset automatically, on a timer, regardless of whether processing actually finished	Risky — the offset can advance <i>before</i> processing completes; if the consumer then crashes, those events (e.g. offsets 30–40) are silently lost forever, since Kafka now thinks they were handled
Manual commit (recommended)	Application explicitly commits — <i>after</i> processing — either synchronously or asynchronously	Safer — but the exact delivery guarantee (at-most-once vs at-least-once) still depends on <i>when</i> relative to processing you choose to commit (see Section 11)

Manually Seeking an Offset

Kafka also allows a consumer to manually rewind or jump to a specific offset — useful for debugging or replaying data:

```
consumer.seek(partition, offset)
```

7. How Consumers Actually Read Data — The Poll / Long-Poll Model

A common misconception is that Kafka **pushes** events to consumers (like the Observer pattern, where subscribers get notified automatically). It does not. Kafka uses a **pull/poll-based model**:

- Consumers periodically **poll** Kafka, passing their last known offset.
- If there's no new data yet, the broker can hold the request open briefly and respond immediately once data arrives — this technique is called **long polling**.
- Consumers are **not notified** by Kafka the moment an event arrives — there's no push mechanism at all.
- This makes each consumer **fully independent** — they can consume at their own pace, on their own schedule (e.g. every 500ms, or every 5 seconds — whatever interval a given microservice chooses).
- This is also exactly why Kafka **naturally handles backpressure** — see Section 12.

Pushing to every subscriber (as classic Pub/Sub does) doesn't scale well when there can be many, independent, differently-paced consumers — the broker would have to actively track and push to every one of them. Polling avoids that coordination overhead entirely, shifting responsibility to each consumer.

8. Replication & the Leader-Follower Model — In Depth

To avoid a single point of failure at the **partition** level (not just the broker level — brokers already get redundancy via having multiple machines), Kafka maintains **replicas** of every partition, controlled by the **replication factor** set at topic creation.

- **Replication factor = 1** → dev/testing only (no real fault tolerance)
- **Replication factor = 2** → small internal systems
- **Replication factor = 3** → industry standard for serious systems
- **Replication factor 4+** → very critical systems that can't tolerate multiple simultaneous broker failures

8.1 Why Replicas Must Live on Different Brokers

If both the "original" partition and its replica sat on the *same* broker, and that machine died, you'd lose both copies at once — completely defeating the purpose of replication. So Kafka deliberately spreads a partition's replicas across **different brokers**.

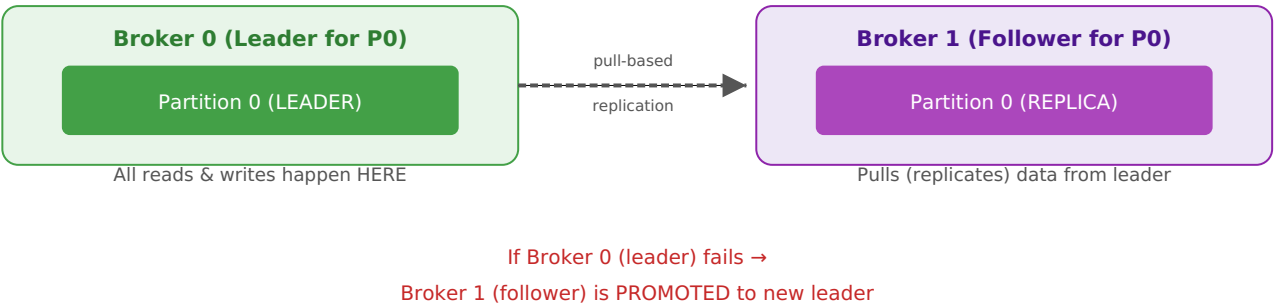


Fig 8.1 — Leader-Follower replication: only the leader serves reads/writes; a follower is promoted on leader failure.

8.2 How Followers Actually Replicate

A natural question: since the follower is a completely different broker/machine from the leader, how does it actually get a copy of the data? The answer, once again, is a **pull-based approach** — just like consumers pull events from partitions, followers **pull** data from the leader partition, at regular intervals, and append it to their own local copy of the log. Because of this, there can be a small **replication lag** between leader and follower, but ignoring that lag, the follower is a complete, faithful mirror of the leader — same events, same offsets.

8.3 Worked Example — 3 Partitions, Replication Factor 2

If a topic is created with **3 partitions** and **replication factor 2**, that means each of P0, P1, P2 gets exactly **one extra copy** (2 copies total per partition). Kafka will typically distribute things so no single broker becomes a bottleneck, and so that a partition's leader and its replica never sit on the same broker:

Partition	Leader Broker	Follower Broker
P0	B1	B2
P1	B2	B3
P2	B3	B1

This layout is just illustrative — the assignment doesn't have to be sequential; Kafka decides based on even distribution of load.

Don't confuse replicas with partitions! Partitions are what give you scalability and parallelism — more partitions means more parallel reads/writes. Replication factor is a completely separate setting whose only job is fault tolerance (surviving a broker going down). Just like databases use master-slave/replica setups purely for resilience, Kafka's replication serves the same single purpose.

8.4 In-Sync Replicas (ISR)

Kafka's metadata tracks something called **ISR — In-Sync Replicas** — representing which follower replicas are currently fully caught up with the leader. This matters for durability guarantees: an acknowledgment can be configured to wait until enough in-sync replicas have confirmed they've replicated a write, not just the leader alone.

9. Kafka Metadata & the Controller

Kafka maintains a **metadata / routing table** describing brokers, topics, partitions, and — crucially — which broker is the **leader** for each partition and which brokers hold replicas. This is managed by the **Kafka Controller**.

- Older Kafka versions used **ZooKeeper** for this role.
- Modern Kafka uses **KRaft** (Kafka Raft) as the built-in controller — no external ZooKeeper dependency.

9.1 What's Inside the Metadata

Conceptually the metadata is a **routing table** structured roughly like this:

```
Kafka Cluster
├─ Brokers: { brokerId → host, port }
├─ Topics
│   └─ topicName
│       └─ Partitions: [ list of partition IDs ]
│           └─ For each partition:
│               - leaderBroker
│               - replicaBrokers: [ list ]
│               - ISR (in-sync replicas): [ list ]
```

9.2 Full Worked Example

```
Topic: "orders"
Partition 0:
  leader   = Broker-1
  replicas = [Broker-1, Broker-2]
  ISR      = [Broker-1, Broker-2]
Partition 1:
  leader   = Broker-2
  replicas = [Broker-2, Broker-3]
  ISR      = [Broker-2, Broker-3]
```

From this metadata, everyone in the system — producers, consumers, and Kafka itself — knows exactly **where a request should go**: all reads and writes for Partition 0 must go to Broker-1 (its leader); for Partition 1, to Broker-2.

9.3 What Happens When a Topic Is Created

1. The **Kafka Controller** creates the metadata entry for the new topic.
2. It assigns the requested number of **partitions** to specific brokers.
3. For each partition, it also assigns the required number of **replicas** (based on the replication factor), and designates a **leader** and **follower(s)** for each.
4. The Controller tries to keep the distribution **even** across brokers so no single broker is overloaded, and ensures replicas of a partition land on *different* brokers.
5. This metadata is persisted in Kafka's **metadata store** (KRaft, or historically ZooKeeper).

Because this metadata rarely changes (leaders/partitions are fairly stable once assigned), producers and consumers **cache** it locally after fetching it once, rather than looking it up on every single request.

10. End-to-End Kafka Flow — Every Step, With Numbers

10.1 Producer Side — Full 7 Steps

1. **1 Connect to any broker.** The producer doesn't yet know who the leaders are, so it connects to *any* broker in the cluster and requests the cluster metadata.
2. **2 Receive & cache metadata.** The producer now knows all brokers, topics, partitions per topic, and — critically — which broker leads each partition. Since this rarely changes, the producer caches it.
3. **3 Decide which partition to send to.** This is entirely the **producer's own responsibility**, not Kafka's:
 - If a **key** is present (e.g. order ID) → `partition = hash(key) % num_partitions`. Worked example: order ID = 123 → `hash(123) = 4` (say) → `4 % 3 = 1` → send to Partition 1. The same key will always map to the same partition, preserving per-key ordering.
 - If no key → the producer uses a simple **round-robin** across the topic's partitions.
4. **4 Find the leader broker.** Using the cached metadata, the producer looks up which broker is the leader for the chosen partition (e.g. Partition 1 → leader is Broker 2), and sends the event **directly** to that broker. No load balancer or proxy is required in between — the metadata already gives the producer everything it needs.
5. **5 Leader appends & assigns offset.** The leader broker appends the record to its local partition log (remember: Kafka only ever appends — no updates/deletes) and assigns it the next sequential **offset**. This offset assignment is done entirely by **Kafka's broker**, never by the producer.
6. **6 Followers replicate.** Follower brokers **pull** the new record from the leader and append it to their own local copy of the log.
7. **7 Acknowledge.** Followers send acknowledgments back; the leader updates the ISR list accordingly, and finally acknowledges the producer that the write succeeded.

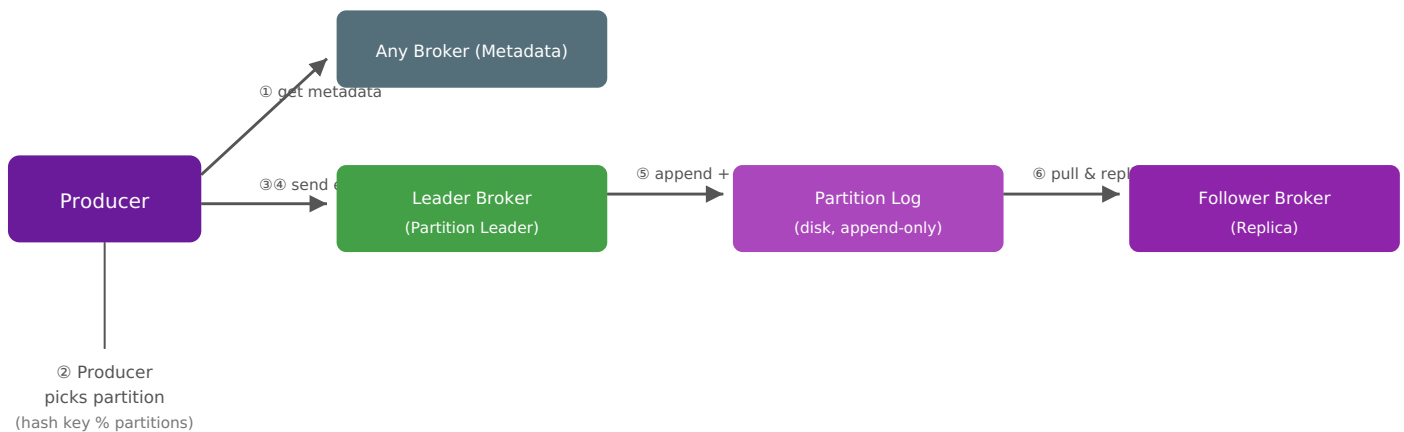


Fig 10.1 — The seven-step producer write path in Kafka.

10.2 Consumer Side — Full Flow

1. Consumer fetches metadata to find which broker leads the target partition (very similar process to the producer).
2. Consumer fetches its **last committed offset** from `__consumer_offsets` for (its consumer group, topic, partition).
3. Consumer **polls** the leader broker for events after that offset.
4. Consumer processes the events, then explicitly **commits** the new offset back to Kafka (assuming manual commit / at-least-once, the default).

Q: Does the producer need a load balancer to know where to send data?

No. Because the producer already has the full metadata (which broker leads which partition), it connects **directly** to the correct leader broker. No load balancer or proxy is required in the middle for this purpose.

11. Handling Wrong / Bad Data in Kafka

Kafka only supports **append** — there is no update or delete of individual records (this is a core reason Kafka is **not a database**). So how do we deal with a bad/incorrect event that's already been written?

- 1. **Compensating events.** Publish a brand-new corrective event rather than editing the old one.

Example: Suppose a wrong event was published: `{orderId: 501, status: "shipped"}` — but it should have said "cancelled." Instead of editing that record, publish a new event: `{orderId: 501, status: "cancelled", correction: true}`. Downstream consumers process this later event and arrive at the correct final state.

- 2. **Validate before producing.** Perform schema validation, required-field checks, and type checks *before* the event ever reaches Kafka — this stops most bad events from entering the log at all.
- 3. **Dead Letter Queue (DLQ).** If, even after validation, a consumer receives a malformed event it simply cannot process, it's routed to a separate **DLQ topic**. The DLQ collects malformed data, unexpected values, and "poison messages," so they can later be **inspected, fixed, and replayed** rather than silently blocking or crashing the consumer.

Kafka is **not a database**. It has a retention period for durability, but you cannot use it for CRUD operations — there is no update/delete of individual records. Data is not retained "forever" the way persistent database storage is — only for the configured retention window.

Old data isn't a growing, unbounded problem either — once an event's **retention period** (whichever hits first: time-based `retention.ms` or size-based `retention.bytes`) expires, it's automatically removed, **whether or not anyone consumed it**. This is an automatic "expiry," not a manual delete operation, and it's what keeps the log from growing forever.

12. Delivery Guarantees — Full Walkthroughs

How and when the offset is committed relative to processing determines the delivery guarantee:

Guarantee	How it works	Trade-off
At-most-once	Offset is committed as soon as events are fetched , before processing completes.	If the consumer crashes mid-processing, those events are lost — they'll never be retried, since Kafka now believes they were already handled.
At-least-once (<i>Kafka's default</i>)	Offset is committed only after processing finishes successfully.	If a crash happens after processing but before the commit call, the same events may be processed twice (duplicates) — but they are never silently lost.
Exactly-once	Uses Kafka transactions — processing + offset commit are wrapped as one atomic unit; a failure anywhere rolls the whole thing back.	Most complex to implement and coordinate; used when duplicates are truly unacceptable (e.g. financial transactions).

12.1 At-Most-Once — Full Walkthrough

Say the offset for a partition/consumer-group pair is **17**. The consumer fetches events 17→24. **Immediately upon fetching** — before doing any actual processing — the offset is updated to 24. Now suppose the consumer crashes while processing those events. Since the offset was already advanced to 24, the next time it starts up, it will fetch from 24 onward — the events between 17 and 24 are simply **gone**, never to be retried. Hence: "at most once" (zero or one time, never more).

12.2 At-Least-Once — Full Walkthrough

Say the offset is **5**. The consumer fetches events 5→10. This time, the offset is *not* updated yet. The consumer finishes processing all of 5→10 successfully — but crashes right before it can commit the new offset (10). On restart, it asks Kafka for its offset and still gets **5** (since the commit never went through), so it fetches 5→10 *again* and reprocesses them — even though they were already fully handled once. This means these events got processed **twice** — hence "at least once" (one or more times, never zero).

12.3 Exactly-Once — Full Walkthrough

This uses **Kafka transactions** (conceptually similar to database transactions). All of "process the batch" + "commit the new offset" is wrapped into a single transaction. If any part fails before the *entire* transaction completes — say processing succeeded but the offset commit didn't — the whole transaction is **rolled back**, as if nothing happened. On restart, the consumer again gets the old offset and reprocesses cleanly from scratch, but this time there's no risk of a "partially committed" inconsistent state, because it's all-or-nothing.

At-most-once



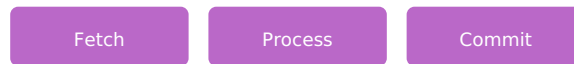
Crash after commit → event LOST

At-least-once (default)



Crash before commit → re-processed (duplicate, not lost)

Exactly-once (Kafka Transactions)



All 3 steps in ONE transaction — all-or-nothing rollback

Fig 12.1 — The three delivery guarantees, driven purely by when the offset commit happens relative to processing.

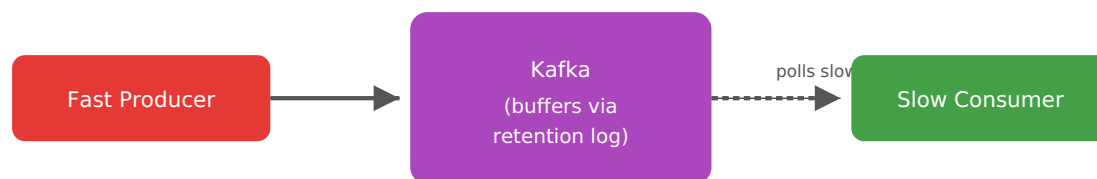
Why Kafka defaults to at-least-once: Kafka prioritizes **never losing data** over avoiding duplicates. Duplicate events can be handled at the application layer using **idempotent writes** and **deduplication logic** — but lost data can never be recovered. So, given the choice, Kafka's default favors duplicates over data loss.

12.4 What Happens During a Consumer Crash (Default Guarantee)

Assuming the default **at-least-once** behavior: say the last committed offset was **100**, and the consumer fetched events up to **110**. While processing, it crashes *before* committing. On restart, it fetches its offset again from `consumer_offsets` and gets **100** again (since the commit never happened) — so records 100-110 are **reprocessed**. This is completely expected and safe under at-least-once semantics, as long as your processing logic tolerates duplicates (e.g. via idempotent writes).

13. Backpressure — The Full Explanation

Imagine a **fast producer** generating a huge volume of events, paired with a **slow consumer** that can't keep up. If nothing mediates this mismatch, you can run into memory fill-ups, thread blocking, JVM crashes, and even data loss on the consumer side. This general class of problem is called **backpressure**, and the mechanism used to prevent it from causing damage.



No push forced onto the consumer — it pulls only what it can handle

Fig 13.1 — Kafka naturally absorbs the mismatch between a fast producer and a slow consumer.

Kafka solves this **naturally**, without any special extra mechanism, purely because of its architecture:

- The fast producer's events simply accumulate in Kafka (up to the retention period) — Kafka doesn't force them onto anyone.
- Because Kafka uses a **pull-based** model (not push), the consumer polls at whatever pace it can handle. Even if the producer is extremely fast and the consumer is very slow, this is **totally fine** — nothing is being force-fed to the slow consumer.
- Multiple independent consumer groups can also each poll at their own speed — one fast, one slow — and this remains fine because of independent offsets per (group, partition).
- Additional strategies on top of this natural handling: a slow consumer can simply choose to **poll less frequently** or in smaller batches.

If Kafka *pushed* events instead (like classic Pub/Sub), a slow consumer would either have to process events it's not ready for, or risk losing them entirely. The pull-based design is precisely what gives Kafka natural backpressure handling.

14. Frequently Asked Questions — Deep Dive

Q: Why is ordering guaranteed within a partition but not across partitions?

Each partition is an independent append-only log, possibly living on a different broker (machine). Coordinating global order across independent, physically separate logs would require constant cross-machine synchronization — killing scalability and parallelism. So Kafka guarantees order only where it's cheap to guarantee it: within a single partition. Since related events (e.g. all events for one specific match, or one specific order) are hashed to land in the *same* partition, this is usually exactly the granularity of ordering you actually need in practice.

Q: Can multiple consumers read from the same partition at the same time?

Only if they belong to **different consumer groups**. Within the same consumer group, only one consumer may read a given partition at any

time — this avoids duplicate work and ordering/offset conflicts within the group. Across different groups, it's completely normal and expected — e.g. topic T with partitions P0/P1/P2, group G1 with C1/C2/C3 (one consumer per partition) and group G2 with C4/C5/C6 (also one consumer per partition) — P0 is legitimately read by both C1 and C4 simultaneously.

Q: What happens during a consumer crash?

With the default at-least-once guarantee: if a consumer fetched events (say offset 100→110) but crashed before committing, on restart it re-fetches from the *last committed* offset (100) — so records 100-110 are **reprocessed**. This is expected and safe as long as processing is idempotent.

Q: Can offsets go backward?

Not automatically — offsets are append-only/increasing by nature. But Kafka does allow a consumer to manually `seek(partition, offset)` to rewind or jump to any point — useful for replay and debugging scenarios.

Q: Does every event have an offset, and who assigns it?

Yes — every event gets one, and it's unique **within its partition** only (not globally — Partition 0 and Partition 1 can both legitimately have an event at offset 5). The **Kafka broker (specifically the partition leader)** assigns the offset sequentially when appending to the log — never the producer. The producer only sends the event; Kafka's broker appends it and stamps the next offset.

Q: What exactly is stored in the consumer_offsets internal topic?

Each record is essentially a map entry: **key** = (consumer group, topic, partition), **value** = the committed offset. E.g. key = (group: "payment-service", topic: "orders", partition: 0), value = committed offset 125. A different partition under the same group/topic gets its own separate entry.

Q: What is back-pressure, and how does Kafka handle it?

See Section 13 for the full explanation — in short, Kafka's pull-based model means a slow consumer never gets force-fed data it can't keep up with; events simply wait in Kafka until it's ready to poll them.

Q: What happens internally when a topic is created?

The Kafka Controller creates metadata, assigns partitions to brokers, designates leaders/followers for each partition (trying to keep load even and replicas on separate brokers), and stores this in the metadata store (KRaft, or historically ZooKeeper). See Section 9.3 for the full step list.

Q: How do you add a consumer group to an existing topic?

You assign a group ID as the consumer group name, and subscribe to the topic:

```
consumer.group.id = "my-consumer-group"
consumer.subscribe(["orders"])
```

After this, it becomes **Kafka's responsibility** to assign specific partitions of that topic to the consumer(s) within the group.

Q: Can a consumer subscribe to multiple topics?

Yes. A single consumer inside a consumer group can subscribe to several different topics at once — e.g. `football_events`, `cricket_events`, and `tennis_events` simultaneously — or even use a **regex pattern** (e.g. subscribing to every topic matching a pattern, or literally every topic in the cluster). Once subscribed, Kafka assigns the consumer specific partitions across all of these topics.

```
consumer.subscribe(Pattern.compile(".*events")) // regex example
```

Q: Can a consumer in a consumer group consume from multiple partitions of the same topic?

Yes — completely normal, and in fact very common, since consumer groups typically have far fewer consumers than a topic has partitions (limited number of microservices vs. potentially many partitions). The only restriction remains: no *other* consumer in the **same** group may also read a partition already claimed by one of its peers.

Q: Can a partition contain events from multiple topics, or can we skip creating new partitions for a new topic?

No on both counts. A partition belongs to exactly **one** topic — never shared/mixed. Every new topic requires its own dedicated set of partitions; you always specify the partition count when creating a topic, and Kafka creates exactly that many new, independent partitions for it.

Q: What does data actually look like on a broker's disk?

Each partition is stored as its own log file plus an index file, typically named with the pattern `topicName-partitionNumber` — e.g. `orders-0`, `orders-1`, `payments-0`, `payments-1`, all potentially sitting side-by-side on the same broker disk (since one broker can host partitions from multiple different topics), but each individual partition folder only ever contains events for its *own* single topic.

15. Why Kafka Handles Massive Reads & Writes

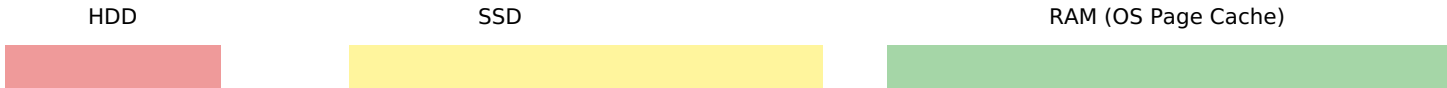
15.1 Why Writes Are So Fast

1. **Append-only log** — no random writes; only sequential disk writes, which are much faster than random I/O (no seeking around the disk).
2. **OS page cache** — writes hit memory first; the actual disk flush happens asynchronously in the background, so the write path isn't blocked waiting on physical disk I/O.
3. **Batching** — producers send **batches**, not single messages one at a time → fewer network round-trips, better compression ratios, and much higher overall throughput.
4. **Parallelism via partitioning** — writes are spread across many independent partitions (potentially on different brokers), enabling true parallel writes rather than a single serialized write path.
5. **Minimal broker-side processing** — the Kafka broker does *not* deserialize the message, does not apply any business logic, and does not filter per consumer. It simply appends, serves, and replicates raw bytes — adding essentially zero extra cost on the write path.

15.2 Why Reads Are So Fast

- **Sequential disk reads** — the same append-only structure that helps writes also helps reads.
- **OS page cache** — hot/recent data is served straight from RAM rather than hitting disk again.
- **No per-message deletion overhead** — since messages aren't deleted right after consumption, there's no cleanup cost interleaved with reads.
- **Independent reads via offsets** — all consumers read independently using their own offsets, so even if a thousand different consumers are reading the exact same data, Kafka only needs to physically read that data from disk **once** and can then serve it from memory **many times over**.

Relative Storage Speed (illustrative, not to scale)



HDD < SSD << RAM — Kafka leans heavily on the OS page cache for both reads and writes

Fig 15.1 — Kafka's reliance on the OS page cache (RAM) is a major reason it can serve enormous read/write volumes cheaply.

16. Kafka vs Pub/Sub — Full Comparison

In classic Pub/Sub (e.g. Redis Pub/Sub), you have **publishers** and **subscribers**. Publishers send messages to the Pub/Sub system; subscribers subscribe to a topic, exactly like Kafka consumers subscribe to a topic. The key architectural difference: when Pub/Sub receives a message, it **pushes** that message to every subscriber that's subscribed to that topic — each subscriber gets its own separate copy.

Aspect	Classic Pub/Sub	Kafka
Delivery model	Push-based	Pull-based (polling)
Retry	Broker-driven (implementation dependent)	Consumer-driven (via offsets)
Back-pressure handling	Hard — broker pushes regardless of subscriber speed	Natural — consumer pulls at its own pace
Message lifetime	Until delivered, then removed	Until retention period expires (time/size based)
Replay	✗ Not possible	✓ Possible
Late-joining consumer	Misses past messages entirely	Can replay from earlier offsets
Consumer state tracking	Broker's responsibility	Consumer's responsibility (via offset)
Fan-out cost	Broker duplicates the message per subscriber — memory/network heavy	One copy stored; many consumers read the same data — cheap, zero duplication
Ordering	Not guaranteed	Guaranteed per partition (not across partitions, for scalability)
Throughput	Lower (per-message routing, subscriber-list maintenance, eager push)	Very high (batching, page cache, sequential I/O, no per-consumer acknowledgment tracking)

A well-known example of classic Pub/Sub is Redis Pub/Sub. Note there can be different Pub/Sub implementations with varying features — the table above describes the "classic" model.

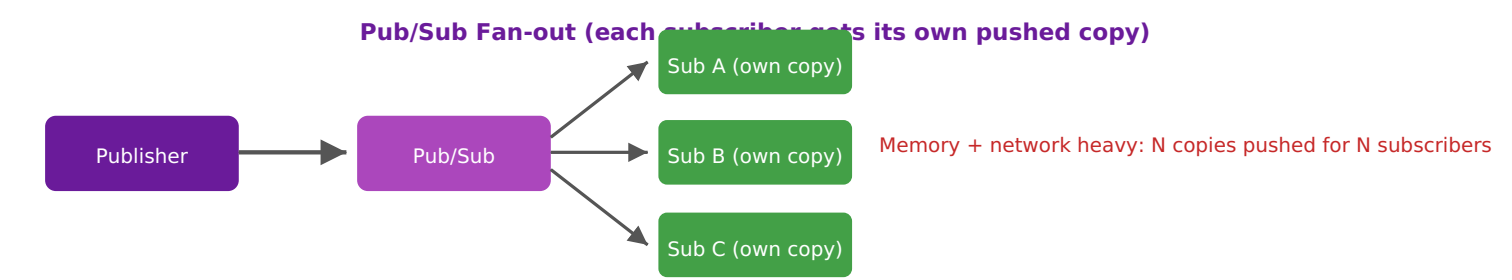


Fig 16.1 — Pub/Sub duplicates the message per subscriber (push fan-out), whereas Kafka stores one copy and lets every consumer independently pull from it.

When to use Pub/Sub? Simple real-time notifications, low-latency push requirements, no need for replay, and a small number of consumers.

When to use Kafka? High write throughput (e.g. batching many writes before a single DB insert), multiple independent consumers/microservices that each need durability and independent pacing, and any scenario needing replay — i.e. most complex, production-grade distributed systems, such as designing YouTube or WhatsApp.

In short: **Kafka ≈ a distributed commit log + Pub/Sub semantics** — strictly more capable than a classic Pub/Sub system, at the cost of slightly more operational complexity.

17. Kafka vs RabbitMQ (Event Stream vs Message Queue) — Full Comparison

RabbitMQ is a well-known example of a classic **message queue / task queue**. Comparing it against Kafka is really a comparison between the **event stream** and **message queue** paradigms discussed in Section 3, now made concrete with two real, popular technologies.

Aspect	Kafka (Event Stream)	RabbitMQ (Message Queue)
What's stored	Events (immutable)	Tasks / messages
Consumers per message	Multiple	Only one
Delivery model	Pull-based (polling)	Push-based
Replay	✓ Possible (retention period)	✗ Not possible
Retention	Time & size based; independent of consumption	Until acknowledged by consumer, then removed
Consumer state	Consumer's responsibility (offset)	Broker's responsibility

17.1 Can Kafka Ever Act Like a Message Queue?

Yes — in the special case where a topic's partition is only ever consumed by **one consumer group with a single active consumer**, effectively only one "listener" gets the data, which resembles the message-queue pattern (since, by the golden rule, no other consumer in that same group could read that partition anyway). But this is best thought of as a theoretical edge case / way of reasoning about it — the vast majority of Kafka's real power and typical usage is as an **event streaming platform** with multiple independent consumer groups reading the same partitions.

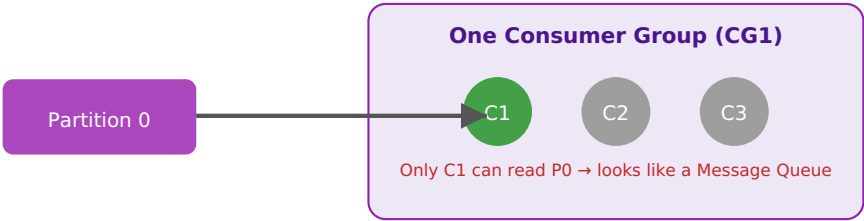


Fig 17.1 — When only a single consumer group (with one active member) reads a partition, Kafka effectively behaves like a message queue for that specific case.

18. Kafka in Real System Design — YouTube Upload Pipeline (Worked End-to-End)

- Bringing everything together, here's how Kafka is actually used inside a real system design — the video upload flow for a platform like YouTube:
- `upload-service` (the **producer**) stores the raw video file in Amazon S3.
 - Once the upload finishes, `upload-service` publishes a `video_uploaded` event to Kafka.
 - Multiple independent microservices — each its own **consumer group**, with its own independent offset — consume this same event from Kafka:
 - `transcoding-service` — converts the video into multiple resolutions: 144p, 360p, 720p, 4K, etc.
 - `copyright-check-service` — checks for copyright issues in the uploaded content.
 - `analytics-service` — updates upload analytics.
 - ...and potentially several more.
 - Each of these consumer groups has a completely independent offset for the same partition — e.g. transcoding-service might be at offset 112, copyright-service at 57, analytics-service at 71 — all progressing at their own pace, polling Kafka on their own interval (e.g. every 500ms or every 5 seconds).
 - A consumer can also become a producer: once `transcoding-service` finishes its work, it can publish a new event, `video_transcoded`, which other downstream consumers can then pick up.

Why Kafka is so valuable here: Without Kafka, `upload-service` would need to directly call `transcoding-service`, `copyright-check-service`, and `analytics-service` individually — creating heavy coupling and putting the burden of orchestrating (and handling failures in) every downstream call on the upload service itself. With Kafka, `upload-service` simply publishes one event and its job is done — every consumer independently decides when and how to process that event. This is exactly how Kafka **decouples microservices** and keeps each one simple and independently scalable.

For the fully detailed system designs (including where exactly Kafka fits among all the other components), refer to complete YouTube and WhatsApp system-design walkthroughs that use Kafka in this same pattern.

19. Code Snippet Appendix

Create a Topic

```
kafka-topics.sh --create \  
  --topic orders \  
  --partitions 6 \  
  --replication-factor 3
```

Background Thread / Executor (Non-Kafka Async Option)

```
executor.submit(() -> {  
    // async task logic here  
});
```

Subscribing a Consumer to a Topic

```
consumer.group.id = "my-consumer-group";  
consumer.subscribe(Arrays.asList("orders"));
```

Subscribing via Regex Pattern (Multiple Topics)

```
consumer.subscribe(Pattern.compile(".*events"));
```

Manually Seeking to an Offset (Replay / Debugging)

```
consumer.seek(partition, offset);
```

Illustrative Event Payload

```
{  
  "topic": "order_events",  
  "event": "order_created",  
  "orderId": 123,  
  "userId": 55,  
  "amount": 499.00,  
  "timestamp": "2026-09-11T10:15:00Z"  
}
```

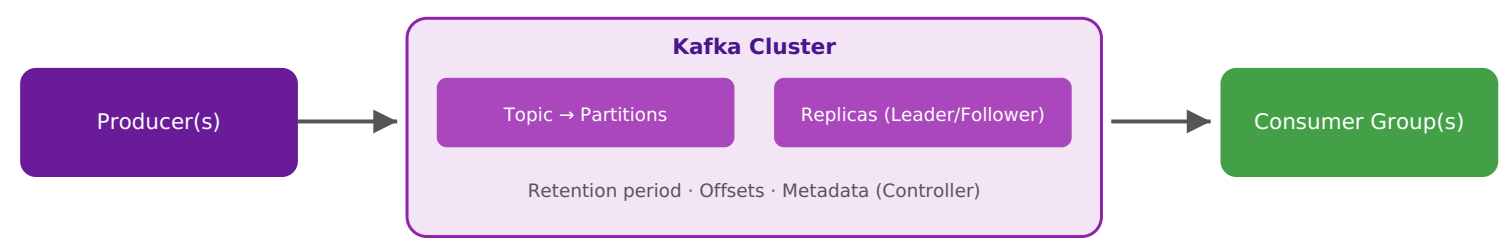
Illustrative Partition-Selection Logic (Conceptual)

```
partition = hash(orderId) % numPartitions;  
// e.g. hash(123) = 4, numPartitions = 3 → 4 % 3 = 1 → Partition 1
```

20. Final Summary Cheat-Sheet

Concept	One-line Definition
Topic	Logical category/stream name for related events (does not store data itself)
Partition	Physical, ordered, append-only log; the actual unit of storage and parallelism
Broker	A server/machine that stores partitions and serves reads/writes
Kafka Cluster	All brokers combined
Producer	Publishes events; decides target partition (hash-of-key or round-robin) — Kafka's responsibility ends at appending & offset assignment
Consumer	Polls and reads events from partitions using offsets; never gets pushed data
Consumer Group	Logical set of consumers; only one member reads a given partition at a time; a consumer cannot exist without one
Offset	Unique, increasing ID of a record within a partition; tracked per (consumer group, partition); acts as a bookmark
__consumer_offsets	Internal Kafka topic that durably stores every (group, topic, partition) → committed offset mapping
Replication Factor	Number of copies of each partition, for fault tolerance (industry standard = 3) — separate concern from partition count
Leader / Follower	Leader serves all reads/writes; followers pull & replicate, and one is promoted on leader failure
ISR	In-Sync Replicas — followers currently fully caught up with the leader
Retention Period	How long events are kept regardless of consumption (e.g. 7-28 days); time or size based, whichever hits first
Delivery Guarantee (default)	At-least-once — no data loss, possible duplicates
Backpressure	Naturally handled via the pull-based polling model — slow consumers are never force-fed data
Dead Letter Queue (DLQ)	A topic that captures malformed/unprocessable events for later inspection and replay
Controller (KRaft/ZooKeeper)	Manages cluster metadata, partition/leader assignment
Kafka append-only nature	No update/delete of individual records — Kafka is not a database

Big-Picture Mental Model



Notes compiled and expanded from a detailed video lecture on Apache Kafka, covering asynchronous processing patterns, message queues vs. event streaming, and the complete internal architecture of Kafka — including worked numeric examples, crash/recovery walkthroughs, and code snippets for self-study and revision.